CURSO AVANZADO DE DESARROLLO WEB

JavaScript Nivel 2: Arquitectura y Asincronismo



Guía técnica de conceptos fundamentales, manipulación avanzada del DOM, patrones de diseño y desarrollo de 3 proyectos reales.

ES6+ Mastery

Proyectos Integradores

MÓDULO 1: SINTAXIS MODERNA

Funciones Flecha y Funciones Anónimas

ES6 Specification

Funciones Flecha (⇒)

Sintaxis compacta: Retorno implícito para expresiones directas.

Lexical 'this': No poseen su propio contexto `this`, heredan el del scope padre.

Diferencias clave: No utilizables como constructores (`new`) ni con `arguments`.

Funciones Anónimas e IIFEs

Definición: Funciones sin identificador usadas en línea.

Uso en Callbacks: Manejo de eventos y métodos iterativos de arrays.

IIFE: Ejecución inmediata para aislar el scope global.

MÓDULO 2: ARQUITECTURA DE CÓDIGO

Recursividad y Clean Code en Funciones

Recursividad

Funciones que se auto-invocan hasta cumplir un caso base. Ideal para estructuras jerárquicas.

Rendimiento y Call Stack

Prevención de desbordamientos de pila (Stack Overflow) y evaluación de algoritmos como Factorial y Fibonacci.

Single Responsibility

Principio donde cada función realiza una sola tarea clara con un nombramiento semántico explícito.

MÓDULO 3: ESTRUCTURAS DE DATOS

Creación y Manipulación Avanzada de Objetos

Patrones de Creación

Objetos Literales: Sintaxis directa con llave-valor.

Constructor Functions: Patrón tradicional de instanciación.

Object.create(): Creación directa con prototipo especificado.

Métodos Estáticos de Extracción

Object.keys(obj) : Arreglo con las claves del objeto.

Object.values(obj) : Arreglo con los valores de las propiedades.

Object.entries(obj) : Pares [clave, valor] en matriz bidimensional.

MÓDULO 4: PROCESAMIENTO DECLARATIVO

Programación Funcional: map(), filter() y reduce()

TRANSFORMACIÓN

map()

Crea un nuevo array con el resultado de transformar cada elemento sin mutar el array original.

FILTRADO

filter()

Evalúa una condición booleana y retorna un subconjunto con los elementos que la cumplen.

ACUMULACIÓN

reduce()

Procesa la lista a través de un acumulador para condensarla en un único valor, objeto o matriz.

MÓDULO 5: INTERFAZ DINÁMICA

Técnicas de Manipulación del DOM

Creación y Eliminación

document.createElement(): Generación de nodos en memoria.

appendChild() / append(): Inserción ordenada en el árbol de renderizado.

removeChild() / remove(): Eliminación limpia de elementos dinámicos.

Modificación y Estilos

Atributos: Manipulación con `setAttribute`, `getAttribute` y `dataset`.

Clases CSS: Gestión de estado con `classList.add / remove / toggle`.

Estilos en línea: Modificación directa de la propiedad `.style`.

MÓDULO 6: INTERACTIVIDAD

Manejo Profesional de Eventos

EventListener & Tipos

Suscripción desacoplada a eventos de teclado (keyup), ratón (click) y formulario (submit).

Propagación de Eventos

Flujo de ejecución a través de Capturing y Bubbling. Delegación de eventos en contenedores padre.

preventDefault()

Interrupción de comportamientos nativos del navegador (ej. recarga de página en formularios).

MÓDULO 7: FLUJOS NO BLOQUEANTES

Introducción al Asincronismo y Promesas

¿Qué es el Asincronismo?

Capacidad de ejecutar tareas pesadas (solicitudes de red, temporizadores) en segundo plano sin congelar el hilo principal de ejecución (Single Threaded Event Loop).

Anatomía de una Promesa

Objeto que representa la eventual finalización o falla de una operación asíncrona.

Estados: Pending → Fulfilled / Rejected

Manejo con `.then()` para resolución y `.catch()` para errores.

MÓDULO 8: SINTAXIS MODERNA Y CONCURRENCIA

Async / Await y Control Concurrente

Async / Await

Azúcar sintáctico sobre Promesas que permite escribir código asíncrono con apariencia síncrona y secuencial.

try...catch

Estructura defensiva de captura de errores para gestionar fallos de red o excepciones en bloques `async`.

Promise.all / Race

`Promise.all()` resuelve múltiples solicitudes en paralelo; `Promise.race()` responde a la primera en finalizar.

MÓDULO 9: PARADIGMAS Y PERSISTENCIA

POO en JS y Web Storage API

Clases y Herencia (ES6)

class & constructor: Definición del molde de objeto.

extends & super(): Jerarquía de clases heredadas.

Static methods: Funciones utilitarias asociadas a la clase.

LocalStorage API

Persistencia de datos clave-valor en el navegador.

JSON.stringify(): Serialización de objetos a texto.

JSON.parse(): Deserialización a objetos JavaScript.

APLICACIÓN PRÁCTICA

Proyectos Integradores 1 y 2

PROYECTO 1

Dashboard de Ventas

Panel analítico para calcular ingresos, productos top y filtrado en tiempo real.

- Encadenamiento de `map()`, `filter()` y `reduce()`.
- Generación dinámica del DOM.
- Persistencia de reportes en `localStorage`.

PROYECTO 2

Catálogo Explorer API

Buscador tipo Netflix con datos asíncronos y comparador concurrente.

- Consumo de API con `async/await` y `try...catch`.
- Consultas simultáneas con `Promise.all()`.
- Delegación de eventos (Event Bubbling).

PROYECTO 3 (PROYECTO FINAL)

Gestión de Tareas Kanban

Sistema visual completo con arquitectura de clases jerárquicas (`Task`, `BugTask`, `FeatureTask`), patrón Singleton, interacción Drag & Drop e hidratación de prototipos.

Reto Principal: Rehidratación de instancias de clase tras la deserialización de objetos planos devueltos por `JSON.parse()` mediante el patrón Factory.



¡Listo para codificar!

Al completar estos 3 proyectos dominarás la manipulación avanzada de JavaScript moderno.